

Lab 11: 2D/3D Graphics Primitives & Basic Transformations

11.1 Introduction to Lab

This lab introduces the essential components and operations used in all computer graphics applications, laying the groundwork for creating virtual environments.

Graphics primitives are the simplest geometric forms (e.g., a point, a line, a triangle, a cube) that are combined to create more complex models, similar to how atoms build molecules. Transformations are the mathematical operations used to position, orient, and size these models within the 3D world space.

Activity Time Boxing

Task No.	Activity Name	Activity time	Total Time
1	Introduction	10 mins	10 mins
2	Walkthrough task	10 mins	10 mins
3	Practice Tasks	30+10 min (Each)	120 mins
4	Conclusion	20 minutes	20 mins
	Total Time		160 Mins

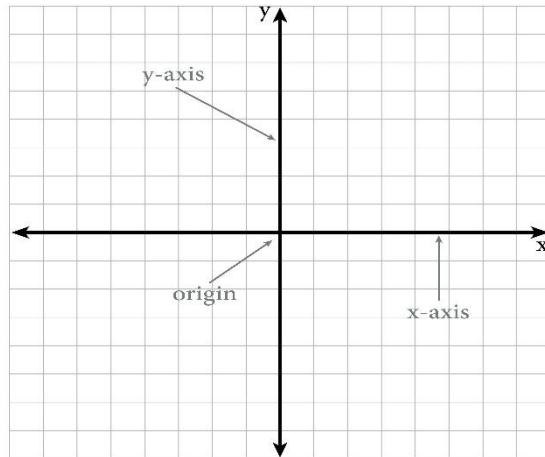
Objective of the Experiment

- Understand how to create and manipulate basic **2D and 3D primitives** (squares, cubes, spheres, etc.) which serve as the foundational elements of any scene.
- Learn to apply **basic geometric transformations, Translation (Position), Rotation, and Scaling** to objects using the **Transform** component.
- Comprehend the concept of how these primitives and transformations act as the core **building blocks** for complex interactive scenes in HCI and computer graphics.

11.2 2D/3D Graphics and Primitives

A. 2D Graphics: The Flat Plane (X and Y Axes)

2D (Two-Dimensional) graphics are confined to a flat plane and are defined using only two spatial coordinates: the **X-axis** (horizontal position or width) and the **Y-axis** (vertical position or height). 2D objects are **flat representations** and inherently lack real depth or volume.

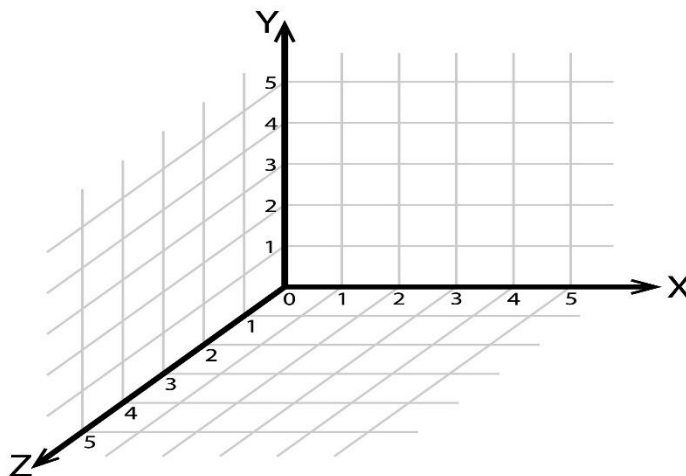


2D Primitives: These are the basic shapes used to construct any 2D scene.

- **Squares** (or Rectangles)
- **Circles**
- **Polygons** (general shapes defined by vertices)

B. 3D Graphics: Adding Depth (The Z-Axis)

3D (Three-Dimensional) graphics introduce a third axis, the **Z-axis** (depth), which allows objects to be represented with **volume** and realistic **perspective**. The Z-axis is perpendicular to the 2D (X-Y) plane.



3D Primitives: These are volumetric shapes that form the building blocks of 3D scenes:

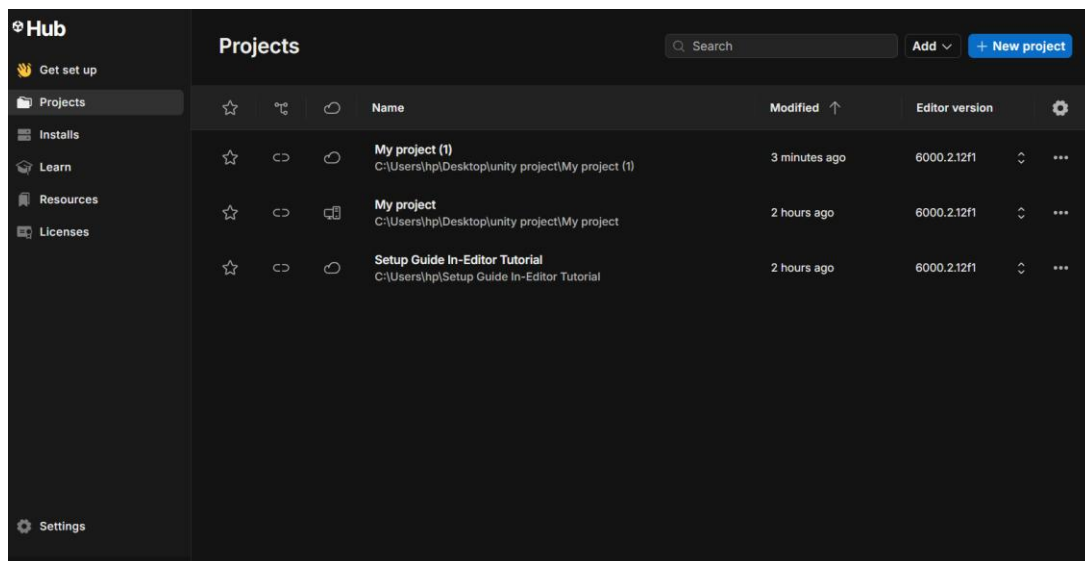
- **Cube**
- **Sphere**
- **Cylinder**
- **Capsule**
- **Plane** (a flat, infinitely thin surface, often used as a floor)

Walkthrough Task

Firstly you need to download Unity by clicking the link <https://public-cdn.cloud.unity3d.com/hub/prod/UnityHubSetup-x64.exe>.

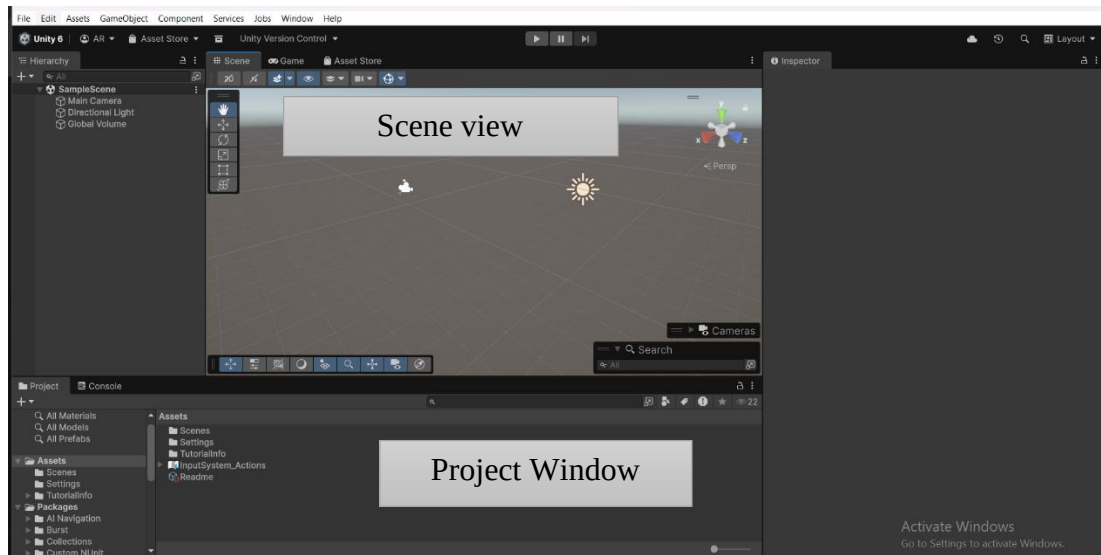
After downloading the unity you need to install the app and open unity hub. Make sure to install unity editor version.

After the editor installs, Go to **Projects**, Click **New Project**, Name your project: **MyFirst2DGame**, Choose the location where the project will be saved, Click **Create Project**.



Then the unity will open

There is **Scene view** where you place objects and adjust the camera view by zooming, moving, or rotating it and then the **project window** where all the assets in your Unity project are stored, such as images, scripts, sounds, prefabs, and materials. It acts like a folder system that lets you organize, view, and manage everything your game uses.

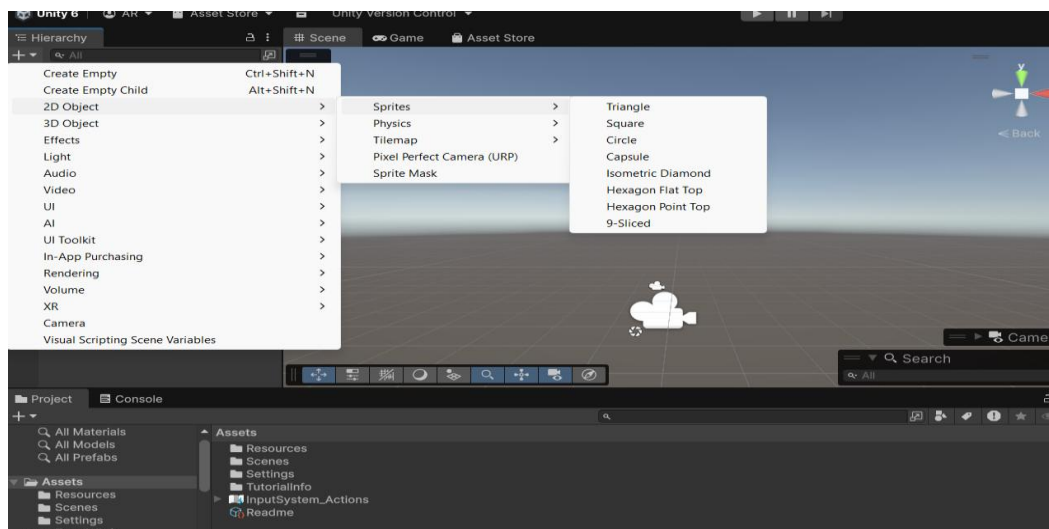


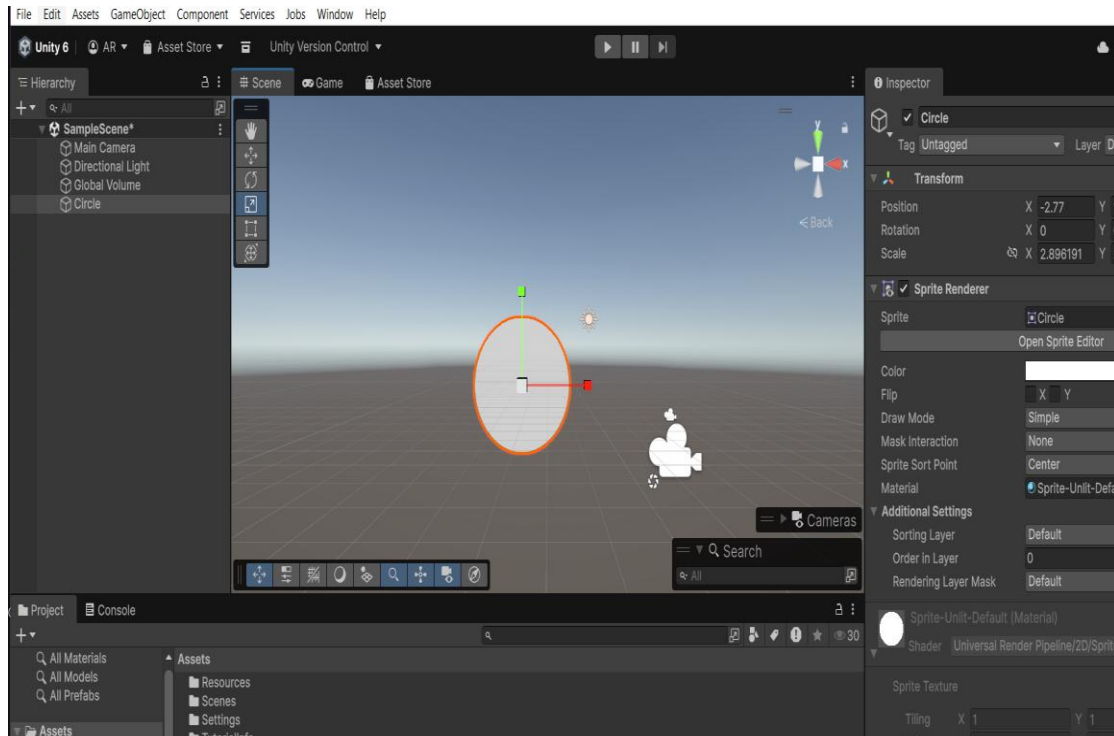
A. Create 2D Primitive

To create a 2D primitive in Unity, go to the Hierarchy, right-click, and choose **2D Object** → **Sprite**. Then select a shape such as a square, circle, capsule, polygon, or even a tilemap for advanced levels. Once selected, the new object will appear in both the Scene and the Hierarchy.

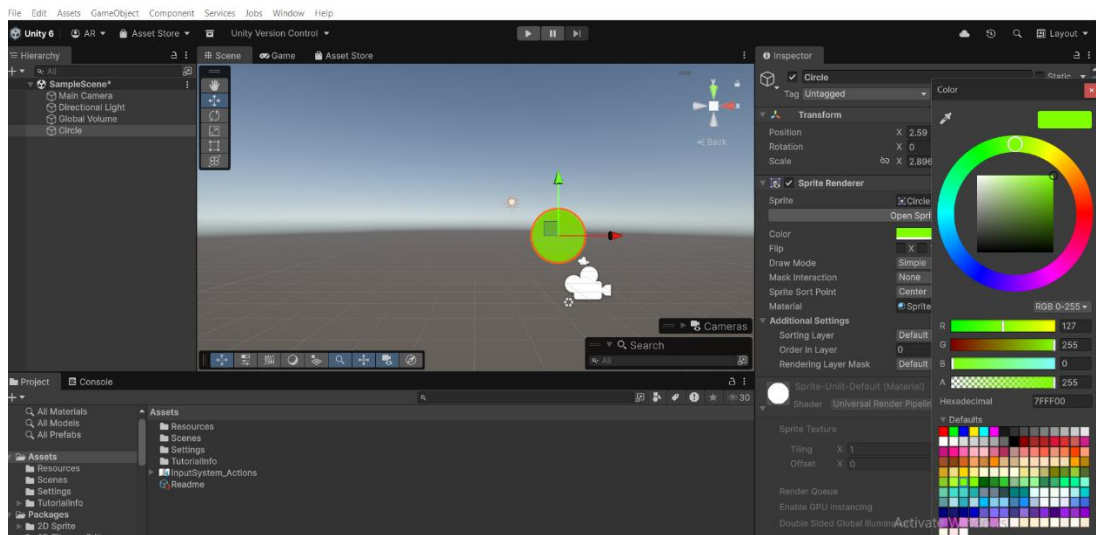
Important Keys: Use **W** to move, **E** to rotate, and **R** to scale a 2D object.

You can also change an object's position, rotation, and scale directly by adjusting its values in the Inspector.



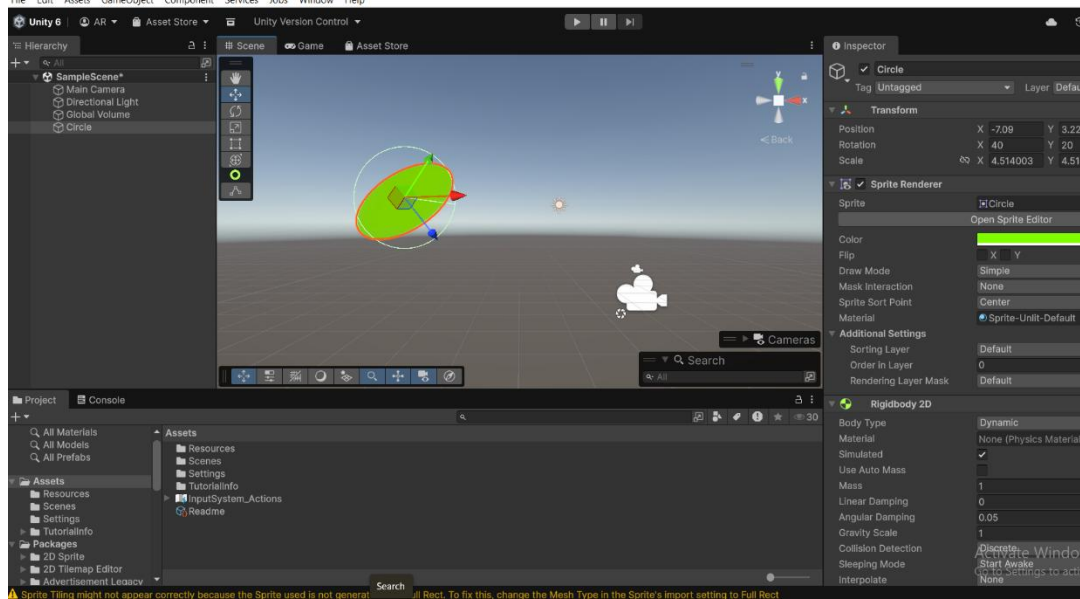


To give **color** to object, Select the object → open **Sprite Renderer** in Inspector → click **Color** → choose any color.



To give **physics behavior** to object (e.g fall because of gravity, **Bounce** if you add material), Select the object, go to **Inspector** → **Add Component** → **Rigidbody 2D**, and add it to enable gravity.

To defines the shape that can collide (boundary). Select the object → Add Component → choose **Box Collider 2D** / **Circle Collider 2D** / **Polygon Collider 2D** to add a collider (green outline will appear).

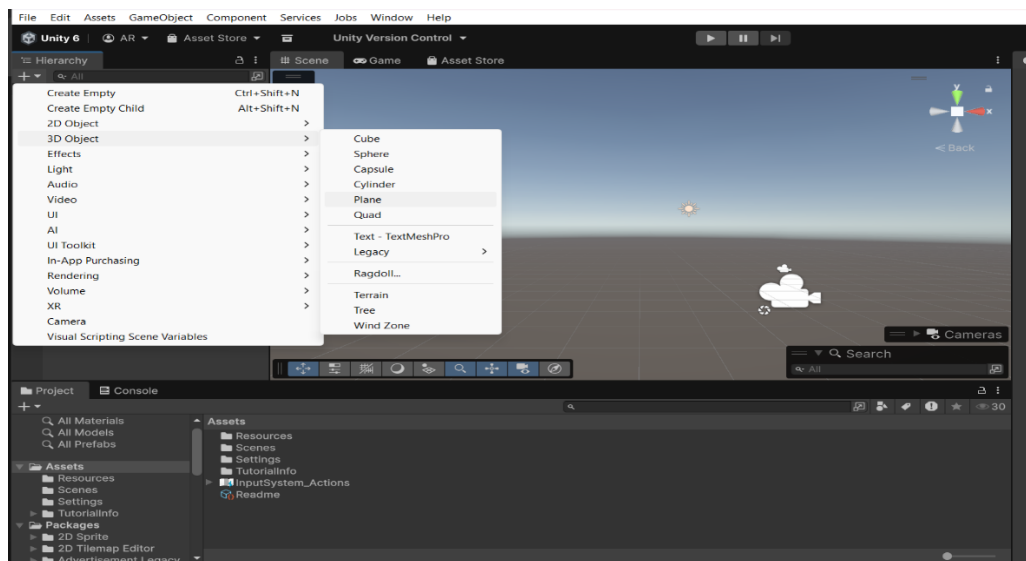


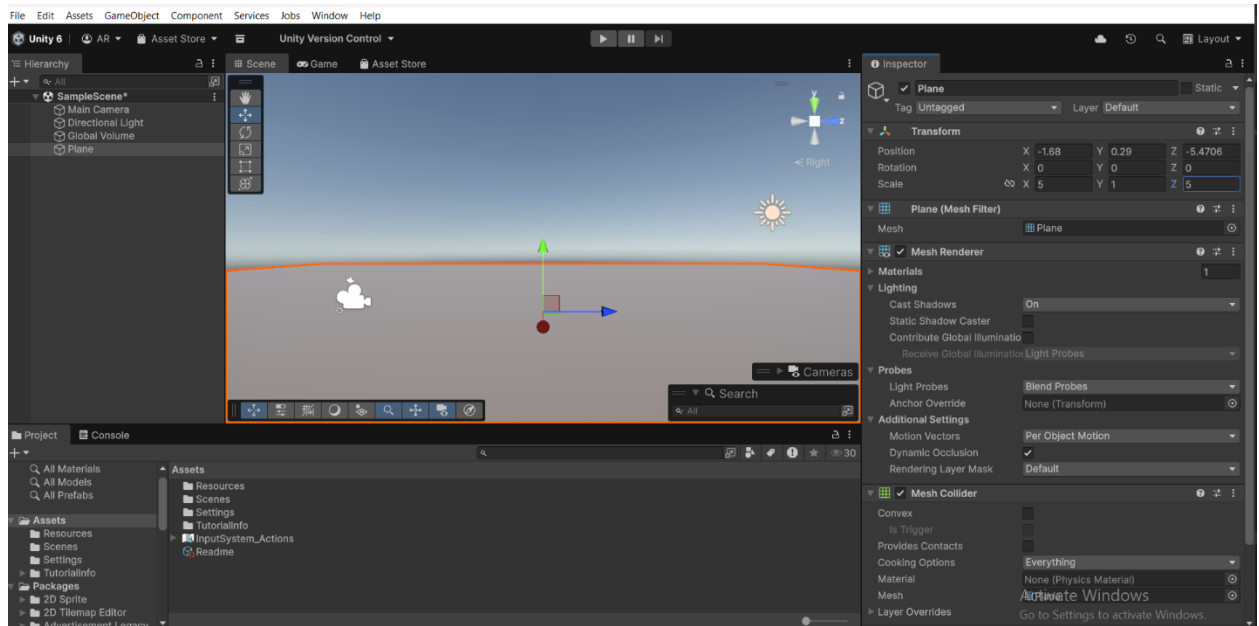
To run your game in Unity, press the Play button at the top of the editor. The screen will show a blue outline to indicate that the game is currently running. When you're done testing, press the Play button again to stop and return to edit mode.

B. Create 3D Primitive

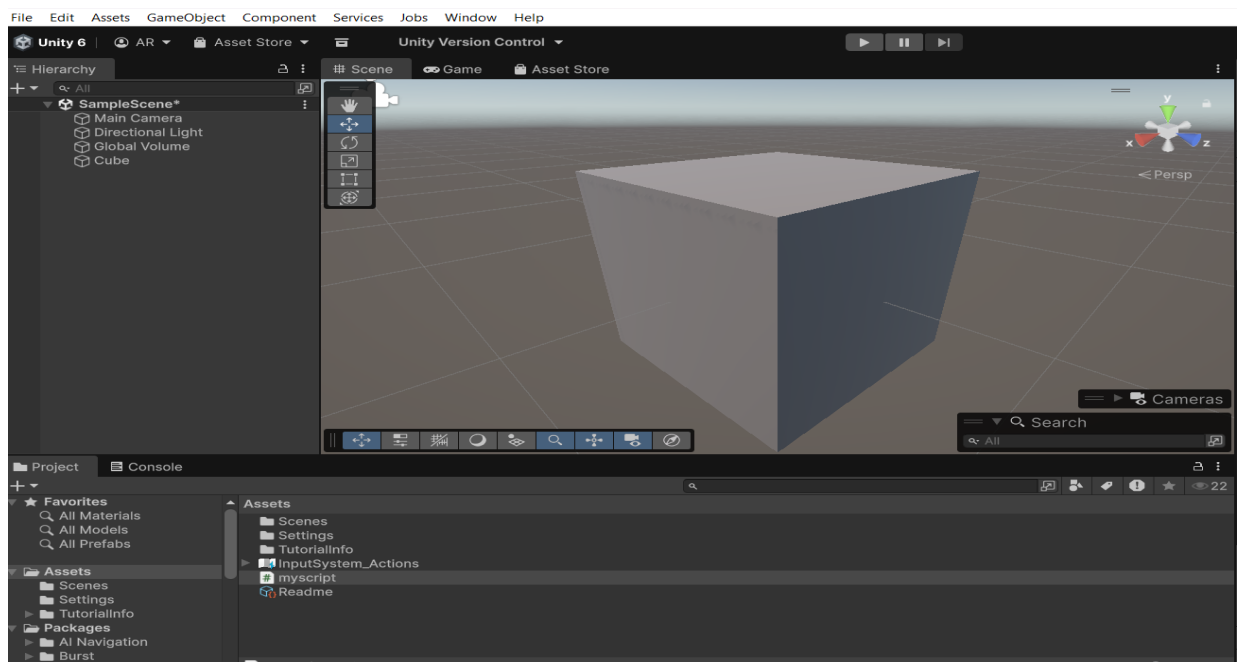
In the Hierarchy, right-click → **3D Object** → **Plane** to add a flat ground, then select the Plane, go to the Inspector, and adjust the **Transform Scale** values (x= 5, y=1,z=5) to make it larger.

We add a plane to serve as the ground or floor for objects in the scene.



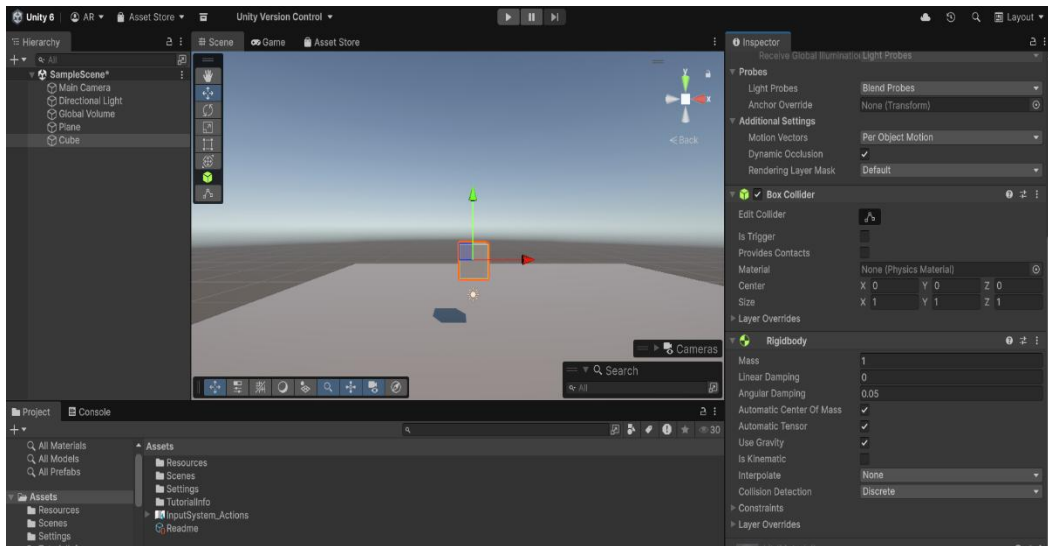


In the Hierarchy, right-click → **3D Object** → **Cube** to add a cube to your scene.

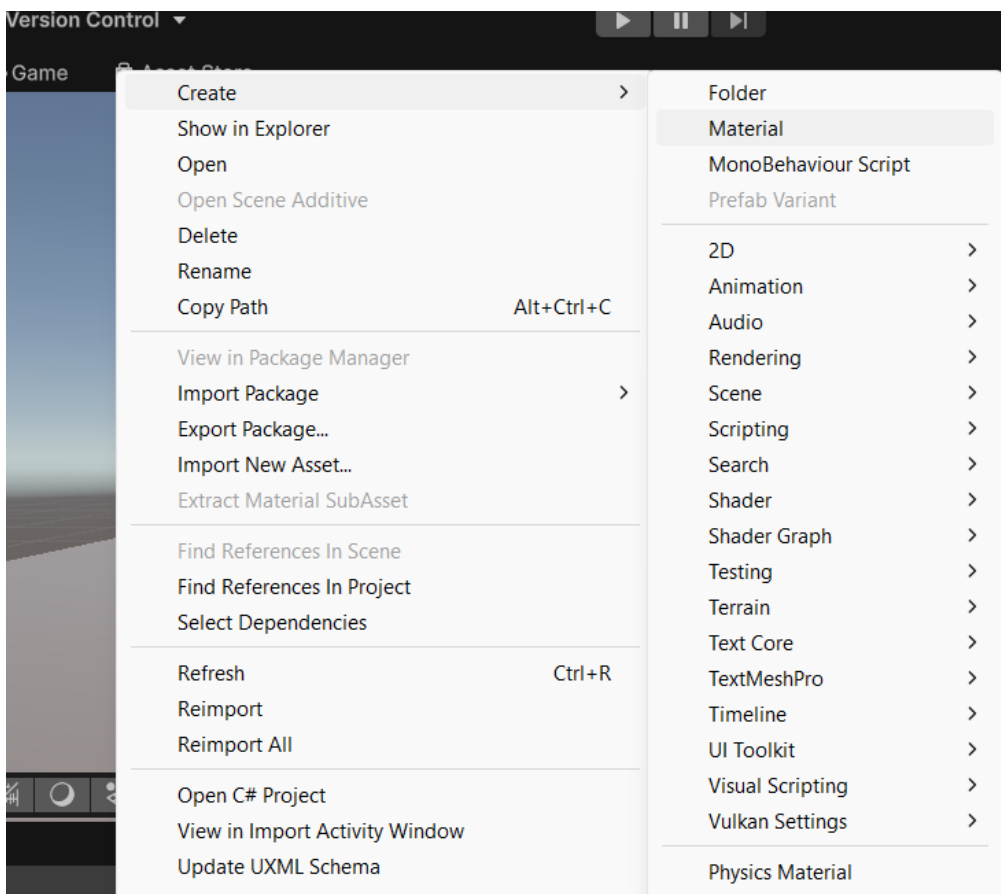


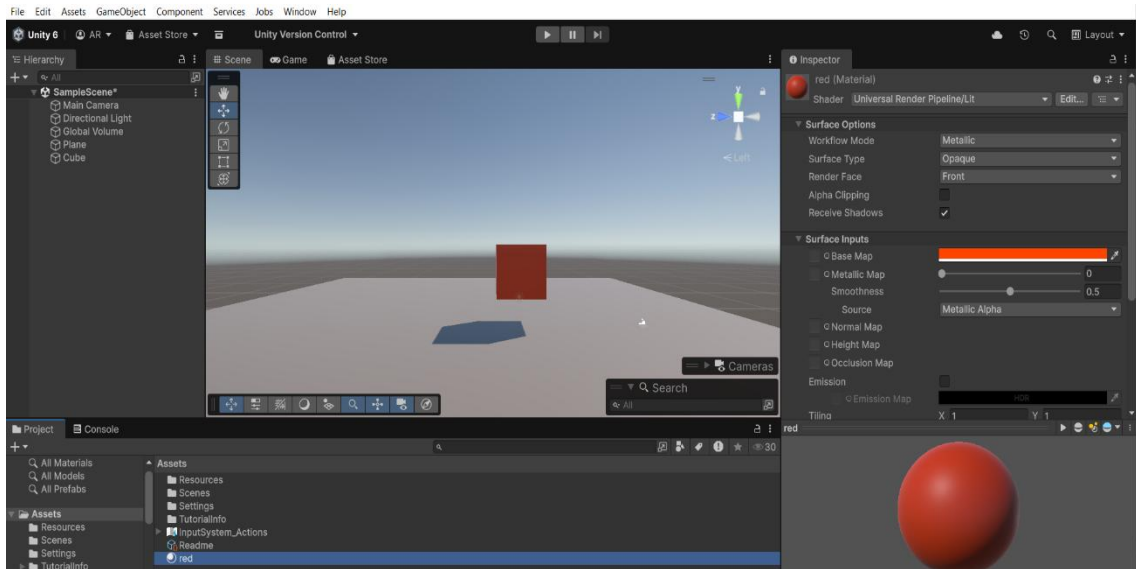
To make it fall, select the Cube, go to **Inspector** → **Transform** → **Position**, and set **X = 0**, **Y = 5**, **Z = 0** to lift it above the plane.

Select the Cube, go to **Inspector** → **Add Component** → **Rigidbody** (not Rigidbody 2D) to add it, so the cube is affected by gravity and can fall.



Create a material in the Project Window → select it and change its **Base Map/Color** in the Inspector → drag and drop the material onto the Cube to apply the color.

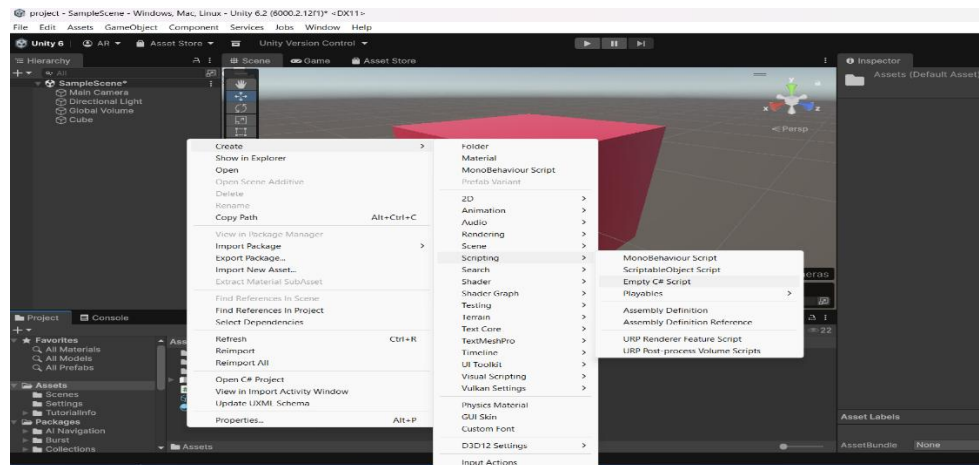




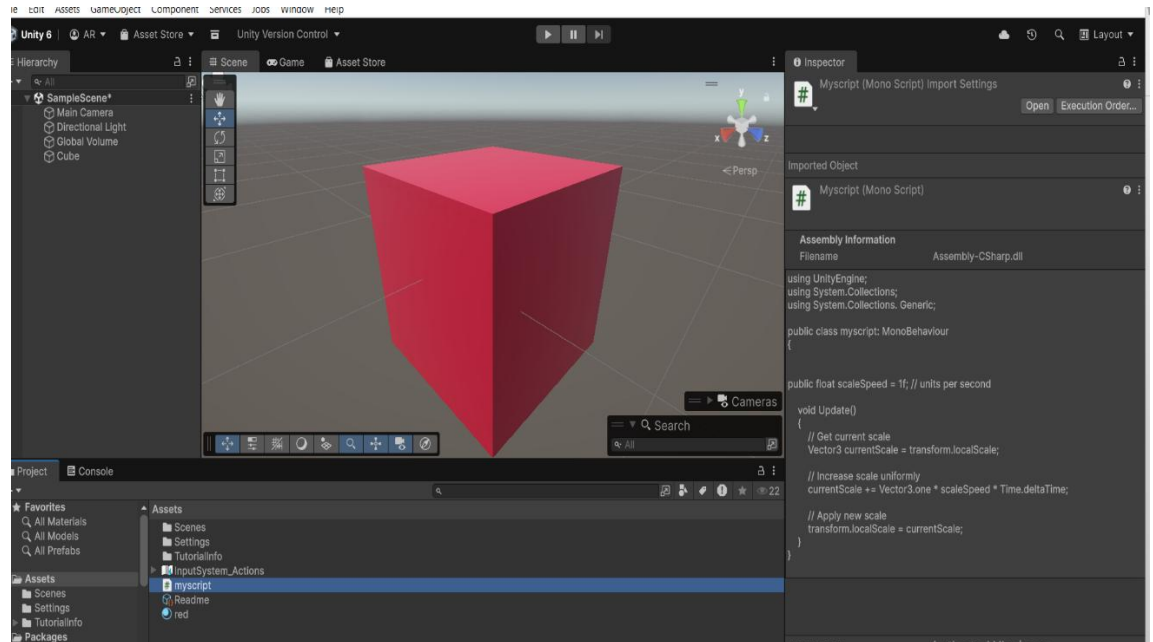
Important Keys: Use **W** to move, **E** to rotate, and **R** to scale a 3D object.

C. Transformations (3D objects)

1. In the Hierarchy, right-click → **3D Object** → **Cube (or Sphere)** → rename it.
2. Right-click in the Project Window → **Create** → **C# Script** → name it `myscript`.
3. Open the script and write code
4. Select the 3D object, go to **Inspector** → **Add Component** → **Scripts** → **myscript** to attach the script or drag and drop the script onto the 3D object in the Hierarchy



3D Rotation



Code:

```
using UnityEngine;
using System.Collections;
using System.Collections.Generic;

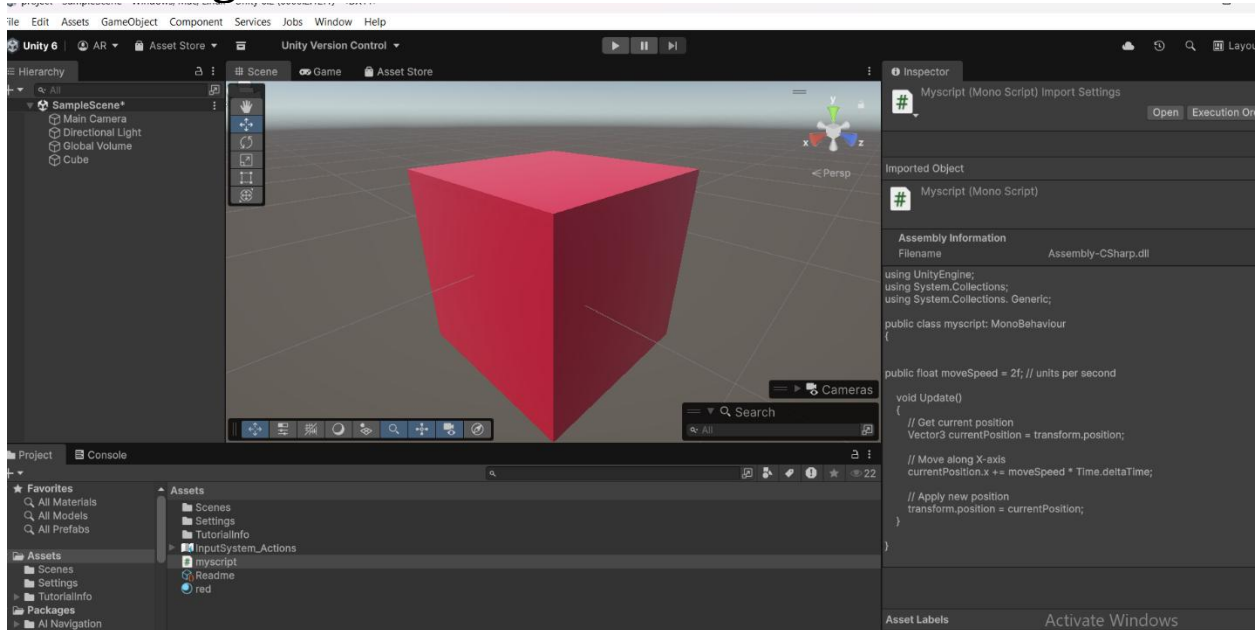
public class myscript: MonoBehaviour
{
    public float rotationSpeed = 90f; // degrees per second

    void Update()
    {
        // Get current rotation
        Vector3 currentRotation = transform.eulerAngles;

        // Add rotation around Z-axis
        currentRotation.y += rotationSpeed * Time.deltaTime;

        // Apply rotation
        transform.eulerAngles = currentRotation;
        //transform.Rotate(Vector3.right * rotationSpeed * Time.deltaTime);
    }
}
```

Positioning



Code:

```
using UnityEngine;
using System.Collections;
using System.Collections.Generic;

public class myscript: MonoBehaviour
{

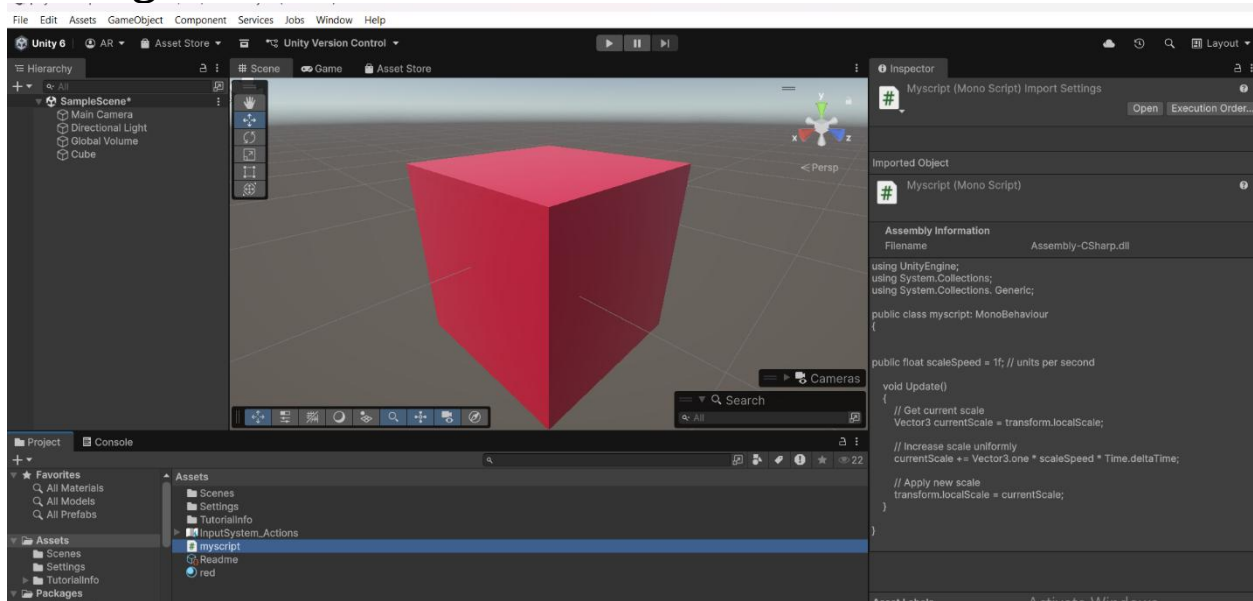
    public float moveSpeed = 2f; // units per second

    void Update()
    {
        // Get current position
        Vector3 currentPosition = transform.position;

        // Move along X-axis
        currentPosition.x += moveSpeed * Time.deltaTime;

        // Apply new position
        transform.position = currentPosition;
    }
}
```

Scaling



Code:

```
using UnityEngine;
using System.Collections;
using System.Collections.Generic;

public class myscript: MonoBehaviour
{
    public float scaleSpeed = 1f; // units per second

    void Update()
    {
        // Get current scale
        Vector3 currentScale = transform.localScale;

        // Increase scale uniformly
        currentScale += Vector3.one * scaleSpeed * Time.deltaTime;

        // Apply new scale
        transform.localScale = currentScale;
    }
}
```

Practice Task

Task 1

(Time =30+10 min)

Create a 2D sprite and a 3D cube with a plane, then make the objects fall and collide with the plane.

Task 2

(Time =30+10 min)

Create a 2D object and write a C# script to rotate, move, and scale it. Press Play to see the object transform automatically in real time. Attach Screen shots in file.

Task 3

(Time =30+10 min)

Create a 3D cube above a plane, then apply physics so that they fall and collide with the plane. Additionally, write a script to make the objects rotate while falling and scale gradually as they move.